Backpropagation, Part 1: What It Is

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. The data and the network . . . 3
4. The steps of backpropagation . . . 5
    4.1 Step 0: initialise the weights and biases . . . 6
    4.2 Steps 1 to 3: one student forward, one loss . . . 6
    4.3 Step 4: update every weight and bias . . . 7
5. Why the error goes backwards . . . 7
    5.1 Another way to see it: who should change, and by how much . . . 8
6. The nine derivatives . . . 10
    6.1 What a derivative tells us here . . . 10
    6.2 The output layer: three derivatives . . . 10
    6.3 The hidden layer: six derivatives . . . 11
    6.4 The pattern . . . 12
7. One update with real numbers . . . 13
    7.1 The nine gradients . . . 14
    7.2 The update . . . 15
8. The full algorithm . . . 16
9. Summary . . . 18
10. Sources . . . 18
11. Key terms . . . 19

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** Derivatives of one variable; Partial derivatives and gradients; Gradient descent; Sigmoid function; Multi-layer perceptron (MLP); Forward propagation; ANN for classification; ANN for regression; Loss functions in deep learning.
- **Leads to:** Vanishing gradient; Memoization; Batch size in Keras; Improving a neural network; Weight initialisation; Optimizers in deep learning; Backpropagation in a CNN; Backpropagation through time (BPTT).

1. Overview

Key point: Backpropagation is the algorithm that trains a neural network. For each **observation** (one record, one row of the data table) it predicts, measures the loss (a number for how wrong the prediction is), then works backwards through the network with the **chain rule** (G-371; a rule that passes a change along a chain of dependent quantities) to find how the loss changes with every weight and bias, and moves each one a small step downhill.

A new network starts with weights that are wrong, so its predictions are wrong. To learn, it must find out, for every weight and bias, whether that number should go up or down, and by how much. The procedure that works this out, starting from the error at the output and moving back towards the inputs, is backpropagation.

Backpropagation (G-247), short for *backward propagation of errors*, is the algorithm used to train neural networks. Given a network and a loss function, it computes the **gradient** of the loss with respect to every weight and bias (the list of numbers saying how much the loss changes per small change of each one); **gradient descent** (G-862; repeated small steps downhill on the loss) then uses that gradient to update them.

Training a network means finding the values of its weights and biases that make its predictions on the data as good as possible. Backpropagation is how we find them.

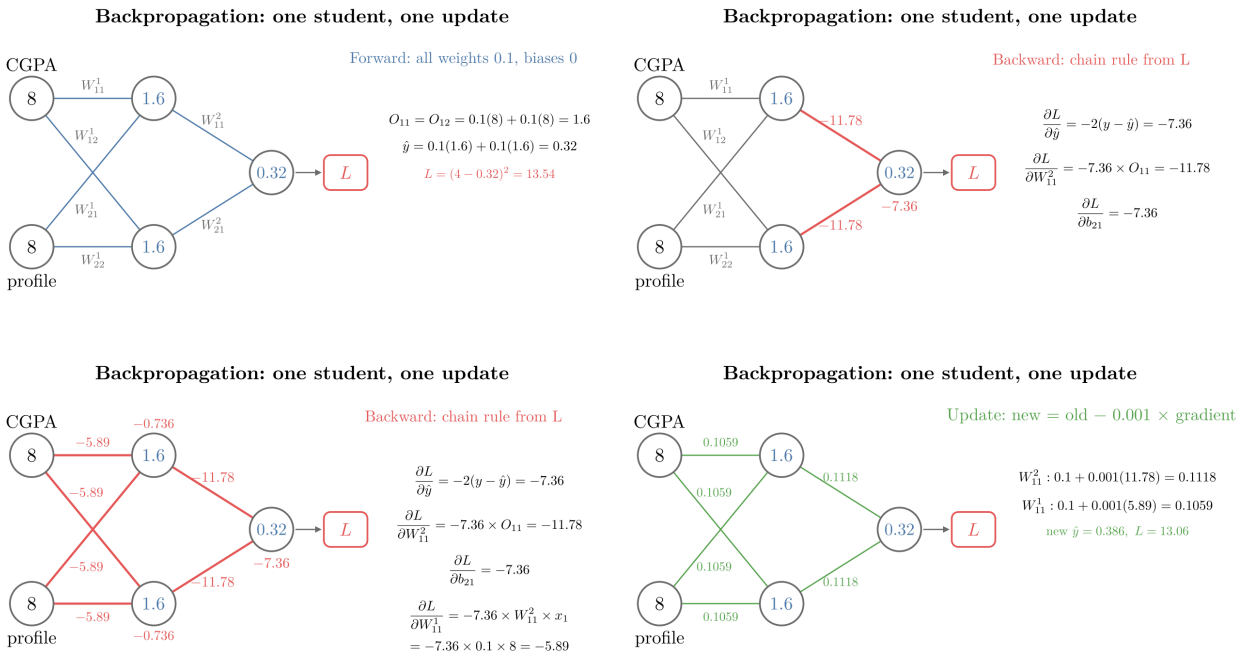


Figure 1: Backpropagation on one student: the forward pass, the loss, the gradients flowing backward onto every weight, and the update

Figure 1 runs the whole algorithm once, for one student. The rest of this Note builds it step by step. Backpropagation takes three Notes:

- this one says **what** it is;
- the regression algorithm in loops runs it in code on a regression and a classification problem;
- why we subtract the derivative explains why the update rule works.

2. Prerequisites

- **Gradient descent** (G-862): the update $w \leftarrow w - \eta\, \partial L/\partial w$ (each parameter w moves against its derivative of the loss L, by the learning rate η).
- **Forward propagation** (G-797): how a network makes a prediction.
- Naming the weights, biases and outputs: W^k_{ij}, b_{kj}, O_{kj} (section 3 shows each with a value).
- The chain rule.

3. The data and the network

> **Key point:** Four students, two **features** (input variables: CGPA, profile score), one **target** (the output we predict: package). A 2-2-1 network with linear activations has 6 weights and 3 biases: 9 trainable parameters.

We predict a student's package (LPA) from their CGPA and profile score, both out of 10:

Student	CGPA x_{i1}	Profile score x_{i2}	Package y_i (LPA)
1	8	8	4
2	7	9	5
3	6	10	6
4	5	12	7

The network has 2 input nodes, one hidden layer of 2 nodes and 1 output node (Figure 2). Predicting a package is a regression problem, so every node uses a **linear activation**: a node outputs its weighted sum plus bias, with no sigmoid.

In the notation for layers, weights and biases, W_{ij}^{k} is the weight from input i to node j of layer k, b_{kj} is the bias of node j in layer k and O_{kj} is that node's output:

- Layer 1 has the weights $W_{11}^{1}, W_{12}^{1}, W_{21}^{1}, W_{22}^{1}$ and the biases b_{11}, b_{12}; its nodes output O_{11} and O_{12}.
- Layer 2 has the weights W_{11}^{2}, W_{21}^{2} and the bias b_{21}; its node outputs $O_{21} = \hat{y}$.

The count is the parameters of layer 1 plus those of layer 2:

$$(2 \times 2 + 2) + (2 \times 1 + 1)$$

$$= 6 + 3$$

$$= 9$$

So the network has 9 trainable parameters.

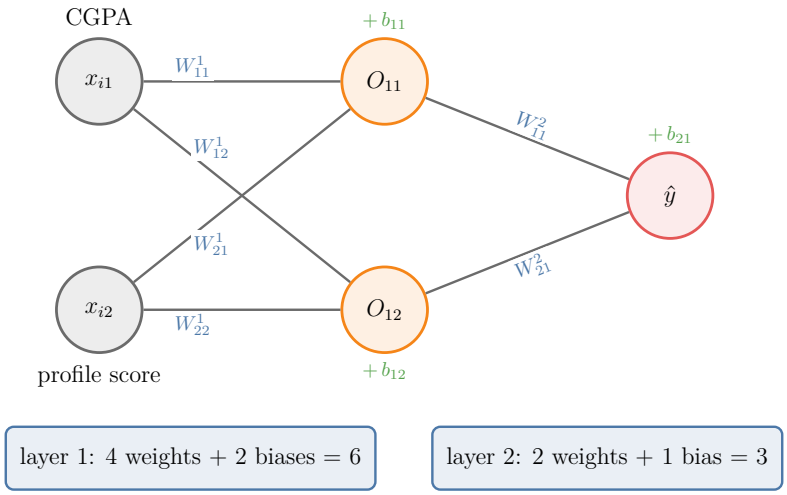


Figure 2: The 2-2-1 network with every weight (blue) and bias (green) named

In Figure 2, every blue label on an edge and every green bias is one of the 9 numbers that training must find.

> **Extra:** Both inputs are scores out of 10, so they sit on the same scale. With IQ (around 80 to 120) next to CGPA, the IQ weights would get gradients about 10 times larger, because a weight's gradient is multiplied by its input (the derivative of O_{11} with respect to W^1_{11} is x_{i1}, section 6). One learning rate would then be too large for the IQ weights or too small for the CGPA weights.

4. The steps of backpropagation

> **Key point:** Step 0: initialise. Then, for each student: **forward propagation** (G-797), loss, update all 9 parameters with gradient descent.

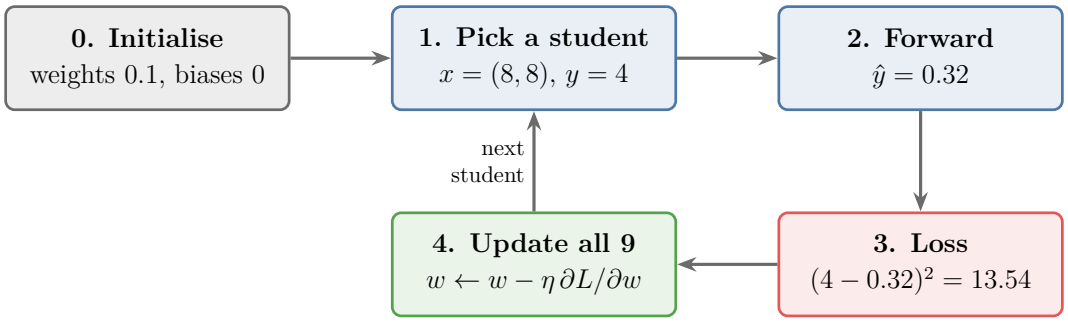


Figure 3: The steps of backpropagation as a loop, with student 1's numbers

Figure 3 is the map for this section: step 0 runs once, then steps 1 to 4 go round once per student.

4.1 Step 0: initialise the weights and biases

> **Key point:** We set every weight to 0.1 and every bias to 0.

Training needs starting values. Common choices are random numbers, or all weights 1 and all biases 0; how well **backpropagation** (G-247) works depends on this choice (why the starting weights matter). To keep the arithmetic easy, we set **every weight to 0.1 and every bias to 0**.

4.2 Steps 1 to 3: one student forward, one loss

> **Key point:** Student 1 gets $\hat{y} = 0.32$ LPA against a real 4 LPA: a loss of 13.54.

1. **Select an observation:** student 1, with $x_{11} = 8$, $x_{12} = 8$ and $y = 4$.
2. **Predict with forward propagation** (see what one node computes):

$$O_{11} = W_{11}^{1} x_{11} + W_{21}^{1} x_{12} + b_{11}$$

$$O_{11} = 0.1 \times 8 + 0.1 \times 8 + 0$$

$$O_{11} = 1.6$$

$$O_{12} = 1.6$$

$$\hat{y} = O_{21} = W_{11}^{2} O_{11} + W_{21}^{2} O_{12} + b_{21}$$

$$\hat{y} = 0.1 \times 1.6 + 0.1 \times 1.6 + 0$$

$$\hat{y} = 0.32$$

3. **Compute the loss.** For regression we use the squared error (see mean squared error):

$$L = (y - \hat{y})^2$$

$$L = (4 - 0.32)^2$$

$$L = 13.54$$

The network says 0.32 LPA where the data says 4. The loss is large because the weights are wrong.

4.3 Step 4: update every weight and bias

> **Key point:** Each of the 9 parameters moves by the learning rate times the derivative of the loss with respect to it.

Every parameter is updated with the **gradient descent** (G-862) rule, where η is the learning rate (the step size) and $\partial L/\partial W$ is the derivative of the loss with respect to the weight W:

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial L}{\partial W_{\text{old}}}$$

$$b_{\text{new}} = b_{\text{old}} - \eta \frac{\partial L}{\partial b_{\text{old}}}$$

For W^2_{11} this reads $W^2_{11} \leftarrow W^2_{11} - \eta\, \partial L/\partial W^2_{11}$, and the same for the other 8. The old value (0.1 or 0) and the learning rate η are known. What we still need are the **9 derivatives**: how much the loss changes when only that one parameter changes a little (the symbol $\partial L/\partial W$ is explained in section 6.1). Computing them is the heart of **backpropagation** (G-247), and it is exactly the definition of Section 1: the gradient of the loss with respect to the network's weights.

5. Why the error goes backwards

> **Key point:** We cannot change y, only $\hat{y}$; $\hat{y}$ depends on three parameters and two hidden outputs, and each hidden output depends on three more parameters. So we fix the error by working back from the output.

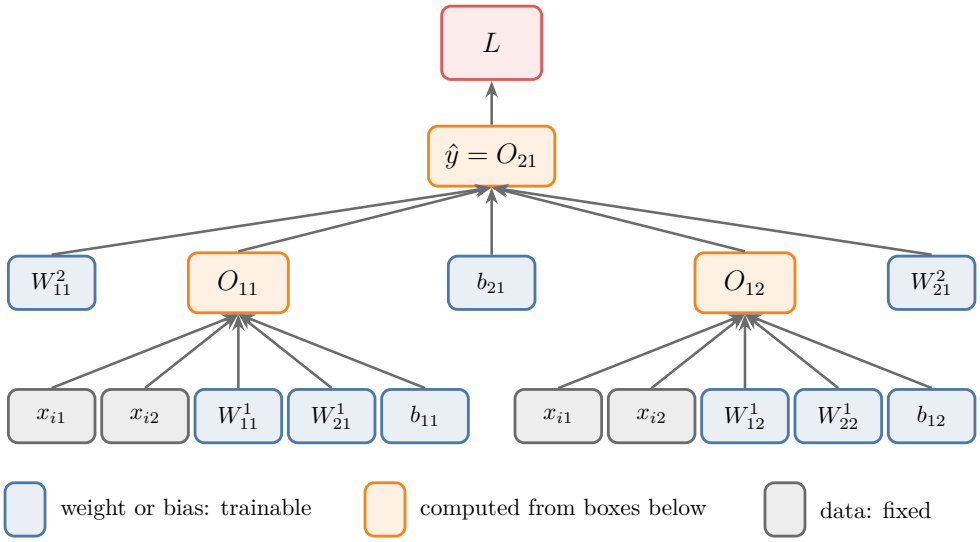


Figure 4: What the loss depends on. To lower L we must change $\hat{y}$, which depends on 5 things; two of them depend on 5 more

The loss $L = (y - \hat{y})^2$ has two parts. y is the true package, fixed by the data. So the only way to lower the loss is to change $\hat{y}$: here, to make it bigger than 0.32.

Figure 4 shows what $\hat{y}$ depends on. Since $\hat{y} = W^2_{11}O_{11} + W^2_{21}O_{12} + b_{21}$, it depends on 5 things:

- W^2_{11}, W^2_{21} and b_{21}: parameters we can change directly;
- O_{11} and O_{12}: outputs of the hidden nodes, which are not parameters themselves.

Each hidden output depends on 5 things in turn. O_{11} depends on the two inputs (fixed data), the weights W^1_{11}, W^1_{21} and the bias b_{11}. O_{12} depends on the inputs, W^1_{12}, W^1_{22} and b_{12}.

Think of a relay team that lost a race: the coach starts with the last runner, sees how much time was lost there, then asks how much of that came from the handover before, and so on back to the first runner. So the loss is reduced by starting at the output and going **backwards**, layer by layer, adjusting the weights and biases in each. The backward direction gives the algorithm its name: the error is propagated backwards.

5.1 Another way to see it: who should change, and by how much

> **Key point:** The output must rise. Its bias, its weights and the hidden outputs can each raise it; within one layer, a parameter that is multiplied by a larger number has a larger effect. The wish for the hidden outputs is passed back one layer, where the same rule applies.

Before any calculus, the backward pass can be told as a list of wishes. For student 1 the network says 0.32 and the data says 4, so the output must rise. The output is $\hat{y} = W^2_{11}O_{11} + W^2_{21}O_{12} + b_{21}$, which gives three ways to raise it.

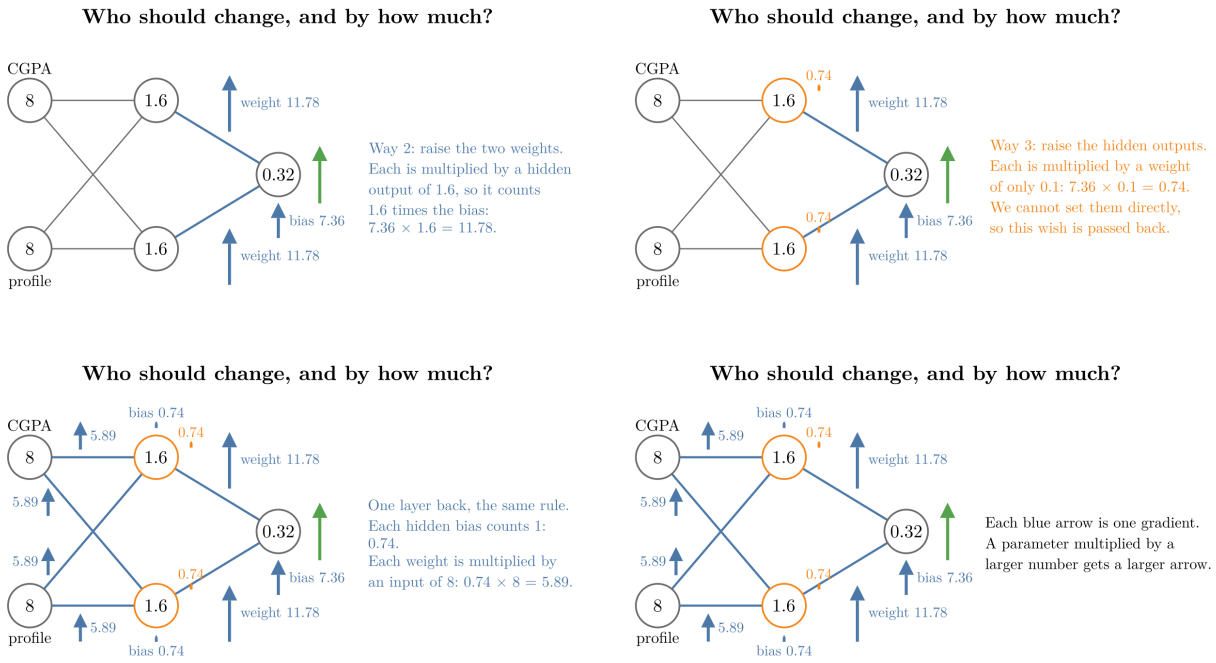


Figure 5: The wishes for student 1 on the 2-2-1 network. Green: the output must rise. Blue arrows: how strongly each weight and bias affects the loss; the number is the size of its gradient. Orange: the wish passed back to the hidden outputs. Idea after 3Blue1Brown, "Backpropagation, intuitively"

In Figure 5, watch the arrows appear from right to left:

1. **Raise the output bias b_{21}.** The bias is added as it is, so one unit more bias gives one unit more output. Its arrow has size 7.36 (section 7.1 computes this number).

2. **Raise the weights W^2_{11} and W^2_{21}.** Each weight is multiplied by a hidden output of 1.6, so one unit more weight gives 1.6 units more output. A weight therefore counts 1.6 times as much as the bias:

$$7.36 \times 1.6 = 11.78$$

   A weight attached to a larger hidden output would count even more.

3. **Raise the hidden outputs O_{11} and O_{12}.** Each is multiplied by a weight of 0.1, so it counts only:

$$7.36 \times 0.1 = 0.74$$

   A hidden output is not a parameter and cannot be set directly. So its wish, "rise, with strength 0.74", is passed back to the layer that produces it.

4. **One layer back, the same three ways.** To raise O_{11}, raise its bias (counts 1: 0.74) or its weights, each multiplied by an input of 8:

$$0.74 \times 8 = 5.89$$

The inputs themselves are data and cannot change, so the passing back stops here.

The arrow sizes 11.78, 7.36, 5.89 and 0.74 are exactly the sizes of the gradients computed in section 7.1. The gradients there are negative: a negative gradient means "raising this parameter lowers the loss", which is the wish of Figure 5. Section 6 now derives the same numbers with the **chain rule** (G-371).

6. The nine derivatives

> **Key point:** Every derivative is a chain: $\partial L/\partial \hat{y}$, times how $\hat{y}$ depends on the next thing, and so on down to the parameter. The first link, $-2(y - \hat{y})$, is shared by all nine.

6.1 What a derivative tells us here

> **Key point:** $\partial L/\partial W_{11}^2$ is how much the loss changes for a tiny change in W_{11}^2.

A derivative dy/dx measures how much y changes when x changes a little (see the derivative). The symbol ∂ marks a partial derivative: the others are held fixed. So $\partial L/\partial W_{11}^2$ asks: if we nudge W_{11}^2, how much does the loss move?

W_{11}^2 does not appear in L directly. A change in W_{11}^2 changes $\hat{y}$, and the change in $\hat{y}$ changes L. The **chain rule** (G-371) handles exactly this: multiply the two rates (see the chain rule):

$$\frac{\partial L}{\partial W_{11}^2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial W_{11}^2}$$

6.2 The output layer: three derivatives

> **Key point:** $\partial L/\partial \hat{y} = -2(y - \hat{y})$, then multiply by O_{11}, O_{12} or 1.

The two factors:

- From $L = (y - \hat{y})^2$: $\partial L/\partial \hat{y} = -2(y - \hat{y})$ (the -1 comes from differentiating $-\hat{y}$ inside the bracket).
- From $\hat{y} = W_{11}^2 O_{11} + W_{21}^2 O_{12} + b_{21}$: only the first term contains W_{11}^2, so the derivative of $\hat{y}$ with respect to W_{11}^2 is O_{11}. In the same way:

$$\frac{\partial \hat{y}}{\partial W_{21}^2} = O_{12}$$

$$\frac{\partial \hat{y}}{\partial b_{21}} = 1$$

So the three derivatives of the output layer are:

$$\frac{\partial L}{\partial W_{11}^2} = -2(y - \hat{y})\, O_{11}$$

$$\frac{\partial L}{\partial W_{21}^2} = -2(y - \hat{y})\, O_{12}$$

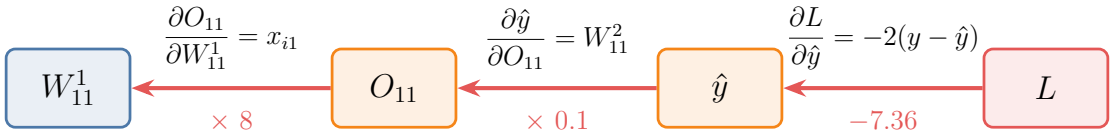
$$\frac{\partial L}{\partial b_{21}} = -2(y - \hat{y})$$

6.3 The hidden layer: six derivatives

> **Key point:** One more link in the chain: first $\hat{y}$ depends on O_{11} through the weight W_{11}^2, then O_{11} depends on W_{11}^1 through the input x_{i1}.

W_{11}^1 is further away. Changing it changes O_{11}, which changes $\hat{y}$, which changes L. The chain has three links:

$$\frac{\partial L}{\partial W_{11}^1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial O_{11}} \cdot \frac{\partial O_{11}}{\partial W_{11}^1}$$



$$\frac{\partial L}{\partial W_{11}^1} = -7.36 \times 0.1 \times 8 = \mathbf{-5.888}$$

first link shared by all 9 derivatives; an output weight stops after two links

Figure 6: The chain from the loss back to W_{11}^1: one factor per link, with student 1's numbers

Figure 6 walks the chain backwards: read it right to left, multiplying one factor per arrow.

- O_{11} appears in $\hat{y}$ only in the term $W_{11}^2 O_{11}$, so:

$$\frac{\partial \hat{y}}{\partial O_{11}} = W_{11}^2$$

$$\frac{\partial \hat{y}}{\partial O_{12}} = W_{21}^2$$

- From $O_{11} = W_{11}^1 x_{i1} + W_{21}^1 x_{i2} + b_{11}$:

$$\frac{\partial O_{11}}{\partial W_{11}^1} = x_{i1}$$

$$\frac{\partial O_{11}}{\partial W_{21}^1} = x_{i2}$$

$$\frac{\partial O_{11}}{\partial b_{11}} = 1$$

  The same holds for O_{12} and its own weights.

Here x_{i1} and x_{i2} are the CGPA and profile score of the student i being processed. The six derivatives of the hidden layer:

Parameter	$\partial L/\partial$(parameter)
W_{11}^1	$-2(y-\hat{y})\, W_{11}^2\, x_{i1}$
W_{21}^1	$-2(y-\hat{y})\, W_{11}^2\, x_{i2}$
b_{11}	$-2(y-\hat{y})\, W_{11}^2$
W_{12}^1	$-2(y-\hat{y})\, W_{21}^2\, x_{i1}$
W_{22}^1	$-2(y-\hat{y})\, W_{21}^2\, x_{i2}$
b_{12}	$-2(y-\hat{y})\, W_{21}^2$

6.4 The pattern

> **Key point:** Every formula is made of numbers we already have after the forward pass: y, $\hat{y}$, the hidden outputs, the current weights and the inputs.

Read all nine together:

- The first factor, $-2(y-\hat{y})$, is the same in all of them. We compute it once, store it, and reuse it. Storing an answer so it is never computed twice is called **memoization (G-1210)** (taught in full later).
- The next factor depends on where the parameter sits: the hidden output it multiplies (O_{11}, O_{12}), or, one layer further back, the weight that carries its node's output forward (W_{11}^2, W_{21}^2).
- The last factor is the input the weight multiplies (x_{i1}, x_{i2}, O_{11}, O_{12}), or 1 for a bias.

After **forward propagation** (G-797) we know y, $\hat{y}$, O_{11}, O_{12}, every weight and both inputs. So all nine derivatives are plain arithmetic, and step 4 can run.

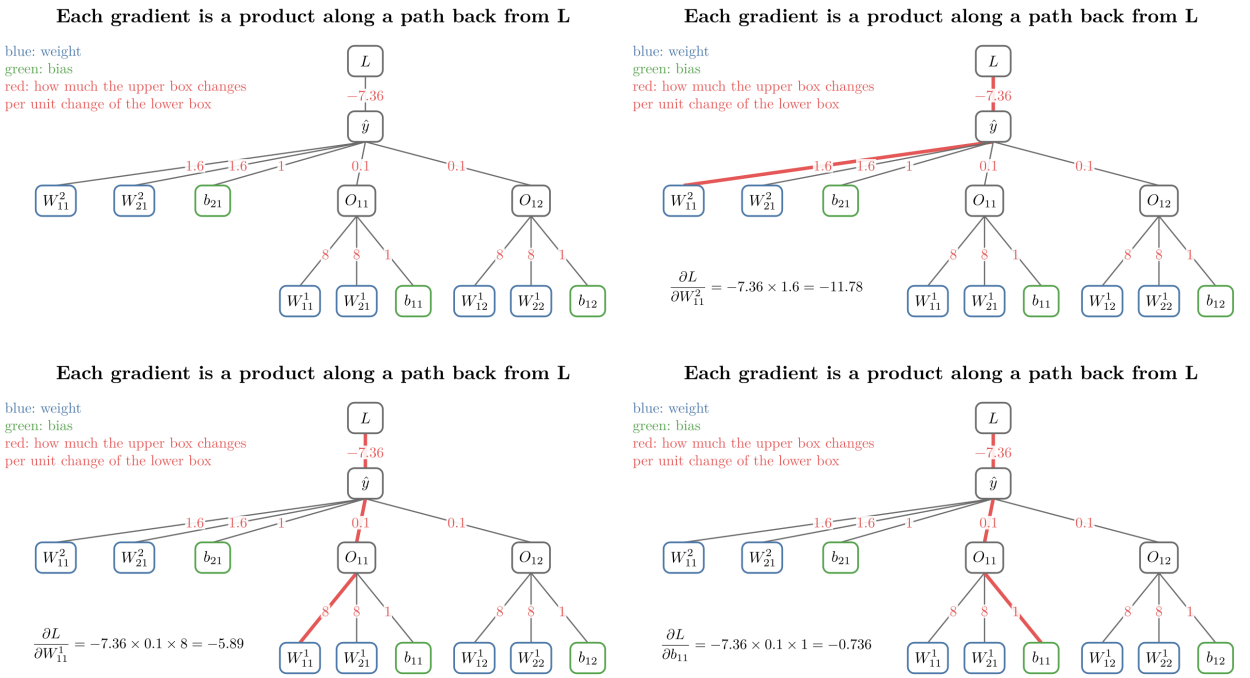


Figure 7: The dependency tree of Figure 4 grown from the loss backwards, with student 1's numbers. The red number on an edge says how much the upper box changes per unit change of the lower box. Multiplying the numbers along the path from L down to a parameter gives that parameter's gradient

Figure 7 shows the pattern as a picture. Watch the tree grow from L downwards, then follow each red path: the path to W_{11}^2 has two edges, the path to W_{11}^1 has three:

$$-7.36 \times 1.6 = -11.78$$

$$-7.36 \times 0.1 \times 8 = -5.89$$

Every path starts with the same edge, -7.36: the shared first factor.

7. One update with real numbers

> **Key point:** For student 1, $\partial L/\partial \hat{y} = -7.36$. With learning rate 0.001, all weights rise slightly and the loss on the same student drops from 13.54 to 13.06.

7.1 The nine gradients

> **Key point:** Output-layer weights: -11.78; hidden weights: -5.89; hidden biases: -0.736; output bias: -7.36.

For student 1: $y = 4$, $\hat{y} = 0.32$, $O_{11} = O_{12} = 1.6$, $x_{11} = x_{12} = 8$, every weight 0.1.

1. **In words:** compute the shared factor once, then multiply by each parameter's own factors.
2. **Formula:** $\partial L/\partial \hat{y} = -2(y - \hat{y})$, then the formulas of Sections 6.2 and 6.3.
3. **Example:**

$$\frac{\partial L}{\partial \hat{y}} = -2(4 - 0.32) = -7.36$$

$$\frac{\partial L}{\partial W_{11}^2} = -7.36 \times 1.6$$

$$\frac{\partial L}{\partial W_{11}^2} = -11.776$$

$$\frac{\partial L}{\partial b_{21}} = -7.36$$

$$\frac{\partial L}{\partial W_{11}^1} = -7.36 \times 0.1 \times 8$$

$$\frac{\partial L}{\partial W_{11}^1} = -5.888$$

$$\frac{\partial L}{\partial b_{11}} = -7.36 \times 0.1$$

$$\frac{\partial L}{\partial b_{11}} = -0.736$$

Parameters	Gradient
W_{11}^2, W_{21}^2	-11.776
b_{21}	-7.36
W_{11}^1, W_{21}^1, W_{12}^1, W_{22}^1	-5.888
b_{11}, b_{12}	-0.736

Every gradient is negative: raising any parameter would lower the loss, which fits a prediction that is far too small. Figure 1 shows these numbers appearing on the network, from the output back to the inputs.

Python: The 9 gradients for one observation.

```
import numpy as np

W1 = np.full((2, 2), 0.1); b1 = np.zeros(2)  # W1[i-1, j-1] = W^1_ij
W2 = np.full(2, 0.1); b2 = 0.0
x, y = np.array([8.0, 8.0]), 4.0

O1 = W1.T @ x + b1                  # forward: O11, O12
y_hat = W2 @ O1 + b2                # forward: y_hat
g = -2 * (y - y_hat)                # dL/dy_hat, shared
dW2, db2 = g * O1, g                # output layer
dW1 = np.outer(x, g * W2)           # dW1[i-1, j-1] = g * W2_j1 * x_i
db1 = g * W2                        # hidden biases
```

`np.outer(a, b)` makes the table of all products `a[i] * b[j]`: here all four hidden-layer weight gradients at once.

Extra: Two independent checks agree to every digit (Notebook). Nudging each parameter by 10^{-6} and measuring the change in the loss gives the same nine numbers (the numerical check of a gradient). TensorFlow's `tf.GradientTape`, which records the forward computation and differentiates it automatically, also returns them. Automatic differentiation is how Keras computes the gradients of any network (see backpropagation and automatic differentiation; TensorFlow guide, Automatic differentiation).

7.2 The update

Key point: W^2_{11}: $0.1 \to 0.1118$; W^1_{11}: $0.1 \to 0.1059$; the prediction rises from 0.32 to 0.386.

With learning rate $\eta = 0.001$:

1. **In words:** subtract the learning rate times the gradient from each parameter.
2. **Formula:** $W_{\text{new}} = W_{\text{old}} - \eta\, \partial L/\partial W$.
3. **Example:**

$$W^2_{11} = 0.1 - 0.001 \times (-11.776)$$

$$W^2_{11} = 0.111776$$

$$b_{21} = 0 - 0.001 \times (-7.36)$$

$$b_{21} = 0.00736$$

$$W_{11}^1 = 0.1 - 0.001 \times (-5.888)$$

$$W_{11}^1 = 0.105888$$

$$b_{11} = 0 - 0.001 \times (-0.736)$$

$$b_{11} = 0.000736$$

Running the same student forward again gives $\hat{y} = 0.386$. The new loss is:

$$(4 - 0.386)^2 = 13.06$$

This is down from 13.54. One small step in the right direction.

> **Extra:** All four first-layer weights got the same gradient, and they still share one value. Because we started every weight at 0.1, the two hidden nodes compute the same thing and receive the same updates, so they stay identical forever and act like a single node. Starting from different (random) values avoids this (Goodfellow et al. 2016, §8.4); starting every weight at the same value shows the problem in a real network.

8. The full algorithm

> **Key point:** Repeat the four steps for every observation (inner loop), then for many epochs (outer loop), until the loss stops falling.

One student moved the weights a little. Training repeats the steps:

1. **Initialise** all weights and biases (here 0.1 and 0).
2. **For each epoch** (one pass over all the data):
   - for each observation of the data (4 here):
     a. **forward propagation** (G-797): predict $\hat{y}$;
     b. compute the loss;
     c. compute the 9 derivatives and update all 9 parameters.
   - compute the average loss of the epoch.
3. **Stop** after a fixed number of epochs, or at **convergence** (G-472): when the loss stops falling.

In the first epoch the four students give losses of 13.54, 21.29, 30.43 and 40.12, an average of 26.35. Each update helps the next prediction a little; the predictions rise from 0.32 to 0.67 over the four students. The losses still grow from student to student because each student's real package is larger (4 up to 7 LPA) while every prediction is still below 1. Running

the outer loop 100 or 1,000 times brings the predictions close to the real packages, as the regression code shows.

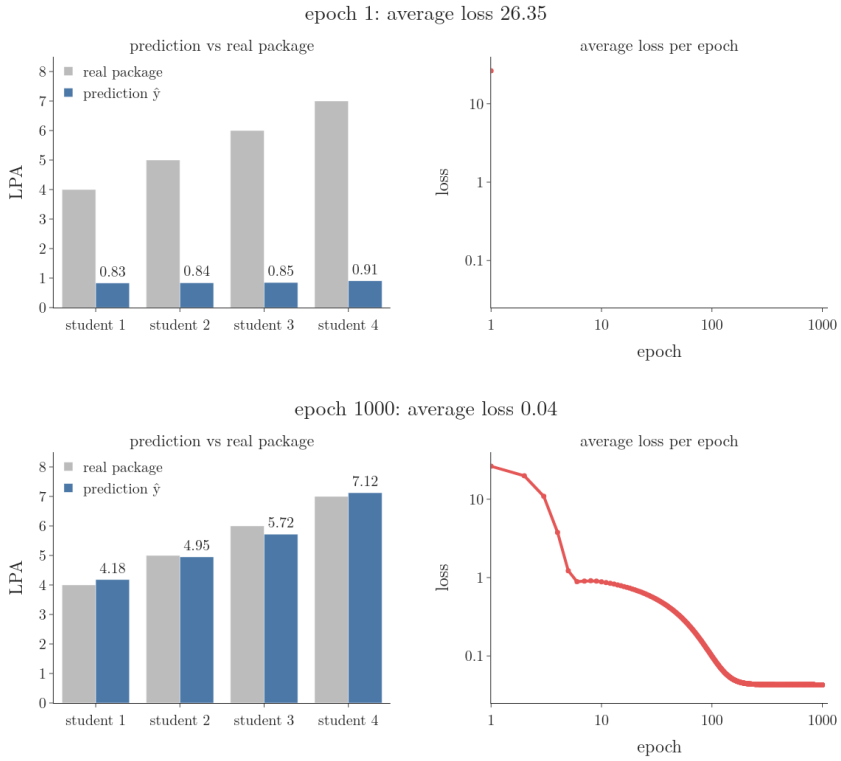


Figure 8: The full algorithm for 1,000 epochs: predictions against real packages (left) and the average loss of each epoch (right)

Figure 8 runs the loop above for 1,000 epochs with learning rate 0.001. Both axes of the loss panel are logarithmic: each gridline is ten times the one below, so the fall from 26 to 0.04 fits on one picture. Watch the blue bars shoot up in the first five epochs, then settle: after 1,000 epochs the predictions are 4.18, 4.95, 5.72 and 7.12 against 4, 5, 6 and 7, and the average loss is 0.04.

Each student gives its own nine gradients, its own list of wishes. If we listened to student 1 alone, the network would learn to fit student 1 alone. Going through all four students lets every student pull the parameters its own way, and the parameters settle where the pulls balance.

Updating after every single observation, as here, is **stochastic gradient descent** (G-1892; see how stochastic gradient descent works); stochastic gradient descent in a network compares it with updating after many observations.

9. Summary

Step	What happens	Student 1
0	Initialise weights 0.1, biases 0	9 parameters
1	Pick an observation	CGPA 8, profile 8, package 4
2	Forward propagation	$O_{11} = O_{12} = 1.6$, $\hat{y} = 0.32$
3	Loss	$(4 - 0.32)^2 = 13.54$
4a	Derivatives by the chain rule	$\partial L/\partial \hat{y} = -7.36$, $\partial L/\partial W_{11}^2 = -11.78$, $\partial L/\partial W_{11}^1 = -5.89$
4b	Gradient descent update, $\eta = 0.001$	$W_{11}^2 = 0.1118$, $W_{11}^1 = 0.1059$; loss 13.06

- **Backpropagation** (G-247) trains a network: it finds the derivative of the loss with respect to every weight and bias, and gradient descent uses them.
- Only $\hat{y}$ can change the loss, and $\hat{y}$ depends on earlier layers, so we work backwards from the output.
- Each derivative is a **chain rule** (G-371) product; the factor $\partial L/\partial \hat{y}$ is shared by all of them, so it is computed once and reused for all nine.
- Within one layer, a parameter that is multiplied by a larger number (a hidden output, an input) gets a larger gradient. Across layers the chain adds factors: the hidden weights multiply an input of 8 but get -5.89, because their chain also passes through a weight of 0.1.
- All the numbers needed come from the forward pass, so the backward pass only multiplies numbers already computed.
- Observations go one at a time; the whole data is repeated for many epochs, because one update moves the loss only a little (13.54 to 13.06 for student 1).

So backpropagation tells every weight and bias whether to go up or down, and by how much: the sign and size of its derivative.

10. Sources

Built from

- CampusX, "Backpropagation in Deep Learning | Part 1 | The What?", YouTube, https://www.youtube.com/watch?v=6M1wWQmcUjQ
- 3Blue1Brown, "Backpropagation, intuitively | Deep Learning Chapter 3", YouTube, https://www.youtube.com/watch?v=Ilg3gGewQ5U (section 5.1: the three ways to raise an output, and passing the wish back)

Other references

- TensorFlow guide, "Introduction to gradients and automatic differentiation" (`tf.GradientTape`).

- Goodfellow, Bengio and Courville, *Deep Learning*, MIT Press, 2016, §8.4 (random initial values break the symmetry between hidden units).

11. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Backpropagation (G-247)	The algorithm that computes the gradient of the loss with respect to every weight and bias by applying the chain rule backward from the output; gradient descent then uses these gradients to train the network.
Gradient (of the loss) (G-863)	How fast the loss changes with each weight and bias, collected in one vector (the partial derivatives of the loss); it points in the direction in which the loss rises fastest.
Chain rule (G-371)	To differentiate a function of a function, multiply the outer derivative by the inner derivative.
Gradient descent (G-862)	Finding the lowest point of a function by repeated small steps downhill.
Forward propagation (G-797)	Passing one row of inputs through the network, layer by layer, to get the prediction.
Memoization (G-1210)	Storing the result of a function call and returning the stored result when the same input comes again.
Convergence (MICE) (G-472)	The point where the fills hardly change between two iterations.
Stochastic gradient descent (SGD) (G-1892)	Gradient descent that uses one random row for every update, so each update is cheap and many updates happen per epoch, at the cost of a noisier path.
Linear activation (G-1089)	An activation that passes a node's weighted sum through unchanged, $f(z) = z$. Used in the output layer for regression.
Initialisation (G-947)	Giving every weight and bias its starting value before training begins; a bad start can cause vanishing or exploding gradients or slow convergence.
Converge (G-469)	To settle at a minimum, with steps becoming negligible.

Term	Meaning
`GradientTape` (G-88)	TensorFlow's tool that records a computation and returns its gradients automatically.